Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 3:
Study and empirical analysis of graph traversal algorithms.

Author:
st. gr. FAF-242 Temciuc Adelina
Verified:
asist. univ. Fiștic Cristofor

Chișinău - 2026

Contents

ALGORITHM ANALYSIS **2**
Objective 2
Tasks 2
Theoretical Notes 2
Introduction 3
Comparison Metric 3
Input Format 3

IMPLEMENTATION **4**
Breadth First Search (BFS) 4
Depth First Search (DFS) 6

Comprehensive Comparison **8**
BFS vs DFS — Sparse Graphs 9
BFS vs DFS — Dense Graphs 10
Key Observations 11

Conclusion **11**

ALGORITHM ANALYSIS

Objective

Study and analyze the performance of graph traversal algorithms, Depth First Search (DFS) and Breadth First Search (BFS) through empirical evaluation.

Tasks:

- Implement the Depth First Search (DFS) and Breadth First Search (BFS) algorithms;
- Determine properties of input data (graphs) for algorithm analysis;
- Choose metrics for comparing the algorithms;
- Perform empirical analysis of the algorithms;
- Present the obtained results graphically;
- Draw conclusions from the laboratory work.

Theoretical Notes:

Empirical analysis provides an experimental approach to evaluate algorithm performance beyond theoretical complexity. It is particularly useful for understanding practical runtime behavior on different input structures and validating theoretical expectations.

Graph traversal algorithms are fundamental in computer science, widely used in network analysis, pathfinding, and scheduling. This laboratory work focuses on two classical traversal techniques:

- **Breadth First Search (BFS)** — explores a graph level by level, visiting all neighbors of the current node before moving to the next depth level. Uses a queue (FIFO) to track nodes to visit. Typical complexity: $O(V + E)$, where V is the number of vertices and E is the number of edges;
- **Depth First Search (DFS)** — explores a graph by following each branch as deep as possible before backtracking. Uses a stack (explicit or via recursion). Typical complexity: $O(V + E)$.

Although both algorithms share the same asymptotic complexity, real execution time may differ due to memory access patterns, cache behavior, recursion overhead, and graph structure (density, connectivity). BFS is generally preferred for shortest path problems in unweighted graphs, while DFS is well-suited for tasks such as cycle detection, topological sorting, and connected component analysis.

The empirical methodology includes:

- Establish the analysis objectives;
- Choose efficiency metrics;
- Define input graph properties (size, density, connectivity);
- Implement DFS and BFS;
- Generate multiple graph datasets;
- Execute experiments and record measurements;
- Analyze and visualize results.

Introduction:

Graph traversal is a fundamental operation for solving a variety of computational problems, including pathfinding, connectivity checking, cycle detection, and network analysis. DFS and BFS are two widely used traversal strategies that differ in their order of exploration, which can lead to differences in practical performance.

While both algorithms share a theoretical complexity of $O(V+E)$, practical execution times depend on factors such as graph representation, size, and sparsity. Dense graphs produce a higher number of edge visits, which may amplify differences in constant factors between the two algorithms. DFS, being recursive in this implementation, also incurs function call overhead that may become significant for large or deeply nested graphs.

Empirical evaluation allows measuring actual performance under controlled experimental conditions, revealing practical differences that theoretical analysis alone may not capture.

Comparison Metric:

The comparison metric used in this laboratory work is execution time $T(n)$ measured in seconds. The `time.perf_counter()` function in Python provides high-resolution wall-clock timing suitable for short-duration measurements.

Each experiment is repeated 5 times per graph configuration, and the average execution time is computed to reduce noise from system fluctuations. The experimental error margin is set at 2.5 milliseconds.

Input Format:

The input data consists of randomly generated undirected graphs with varying structural properties:

- **Number of vertices:** 10, 50, 100, 500, 1000, 2000;
- **Graph density:** sparse ($\approx$ 20% of possible edges) and dense ($\approx$ 80% of possible edges);
- **Graph type:** undirected (edges added symmetrically);
- **Representation:** adjacency list.

Each algorithm is tested against multiple randomly generated graphs for each combination of vertex count and density, producing reliable and averaged runtime measurements.

IMPLEMENTATION

Both DFS and BFS are implemented in Python and evaluated experimentally by measuring their execution time across multiple graph configurations. The actual performance relative to input size depends on hardware specifications, particularly memory and processor characteristics. The experimental error margin is set at 2.5 milliseconds.

Breadth First Search (BFS):

BFS explores a graph by visiting all neighbors of the current node before proceeding to the next depth level. It uses a queue (FIFO) to maintain the frontier of nodes to visit. BFS guarantees that the shortest path (in terms of edge count) is found in unweighted graphs. Its time complexity is $O(V + E)$ and space complexity is $O(V)$ due to the queue and the visited array.

Algorithm Overview:

Listing 1: BFS Algorithm (pseudocode)

```
BFSConnected(adj, src, visited, result):
    enqueue(src)
    visited[src] = True

    while queue is not empty:
        curr = dequeue()
        result.append(curr)
        for each neighbor x of curr:
            if not visited[x]:
                visited[x] = True
                enqueue(x)

BFS(adj):
    visited = [False] * |V|
```

```
    result = []
    for each vertex i in V:
        if not visited[i]:
            BFSConnected(adj, i, visited, result)
    return result
```

Implementation:

```python
from collections import deque

# BFS for a single connected component
def bfsConnected(adj, src, visited, res):
    q = deque()
    visited[src] = True
    q.append(src)

    while q:
        curr = q.popleft()
        res.append(curr)

        # visit all the unvisited
        # neighbours of current node
        for x in adj[curr]:
            if not visited[x]:
                visited[x] = True
                q.append(x)

# BFS for all components (handles disconnected graphs)
def bfs(adj):
    V = len(adj)
    visited = [False] * V
    res = []

    for i in range(V):
        if not visited[i]:
            bfsConnected(adj, i, visited, res)
    return res
```

Figure 1: BFS Implementation in Python

Results:

After executing BFS for each specified graph size and density, the following results were obtained:

```
=== BFS Results ===

Vertices   Time (density=0.2)     Time (density=0.8)
-------------------------------------------------------
10         0.000007               0.000007
50         0.000014               0.000029
100        0.000039               0.000108
500        0.001351               0.004965
1000       0.004991               0.036643
2000       0.055444               0.209486
```

Figure 2: BFS Performance Results

The experimental data confirms the theoretical time complexity of $O(V + E)$ for

BFS. As the number of vertices increases, execution time grows in proportion to the total number of vertices and edges processed. For dense graphs, the growth is steeper because the number of edges scales quadratically with vertex count, while for sparse graphs the growth remains more gradual.

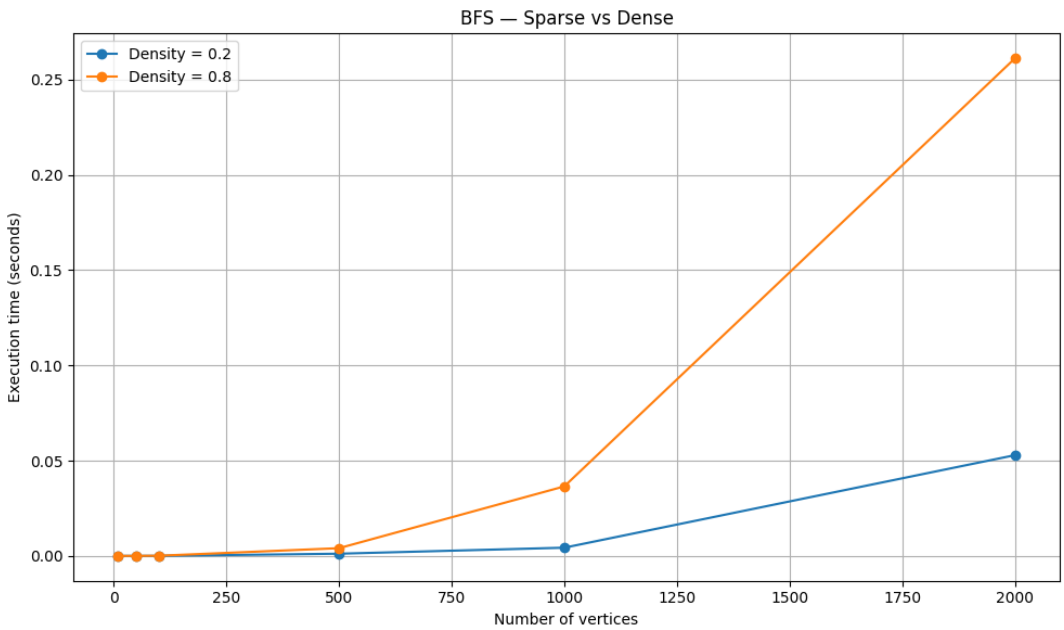

Figure 3: BFS Execution Time Graph

Figure 3 illustrates how BFS scales with input size under two density conditions. The dense curve rises significantly faster due to the larger number of edges visited per vertex. The queue-based iterative nature of BFS avoids recursion overhead, contributing to stable and predictable performance even on large inputs.

Depth First Search (DFS):

DFS explores a graph by following each branch to its maximum depth before backtracking to explore other branches. In this implementation, DFS is realized recursively: each unvisited neighbor is immediately explored before returning. DFS is useful for cycle detection, topological sorting, and connected component identification. Its time complexity is $O(V + E)$ and space complexity is $O(V)$ due to the recursive call stack and visited array.

Algorithm Overview:

Listing 2: DFS Algorithm (pseudocode)

```
DFSRec(adj, visited, s, result):
    visited[s] = True
    result.append(s)
```

```
    for each neighbor i of s:
        if not visited[i]:
            DFSRec(adj, visited, i, result)

DFS(adj):
    visited = [False] * |V|
    result = []
    for each vertex i in V:
        if not visited[i]:
            DFSRec(adj, visited, i, result)
    return result
```

Implementation:

```python
def dfsRec(adj, visited, s, res):
    visited[s] = True
    res.append(s)

    # Recursively visit all adjacent
    # vertices that are not visited yet
    for i in adj[s]:
        if not visited[i]:
            dfsRec(adj, visited, i, res)


def dfs(adj):
    visited = [False] * len(adj)
    res = []
    # Loop through all vertices to
    # handle disconnected graph
    for i in range(len(adj)):
        if not visited[i]:
            dfsRec(adj, visited, i, res)
    return res
```

Figure 4: DFS Implementation in Python

Results:

After executing DFS for each specified graph size and density, the following results were obtained:

```
=== DFS Results ===

Vertices   Time (density=0.2)     Time (density=0.8)
-----------------------------------------------------
10         0.000002               0.000002
50         0.000014               0.000030
100        0.000045               0.000105
500        0.001011               0.004462
1000       0.004993               0.046650
2000       0.047550               0.209201
```

Figure 5: DFS Performance Results

The results confirm the $O(V + E)$ theoretical complexity of DFS. Execution time scales with both the vertex count and graph density. Notably, the recursive implementation incurs Python function call overhead, which may cause DFS to be slightly slower than BFS for equivalent graphs. The recursion limit was raised to 5000 to accommodate larger graphs.

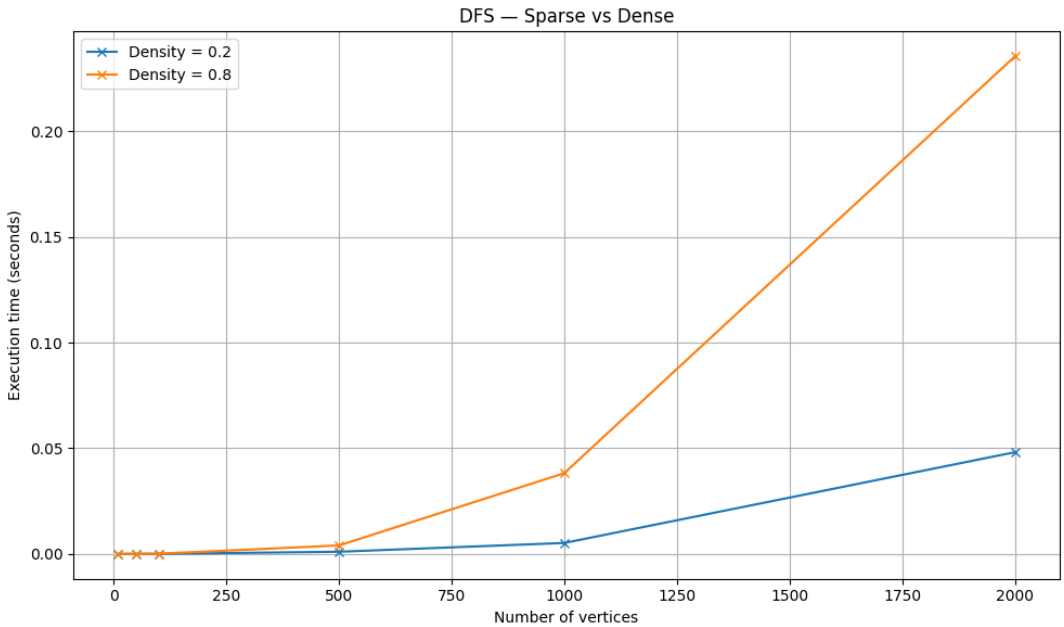

Figure 6: DFS Execution Time Graph

Figure 6 shows a consistent increase in execution time as input size grows. The dense curve grows noticeably faster than the sparse one, consistent with the higher edge count. The recursive nature of DFS introduces additional overhead compared to BFS for large dense graphs, as each recursive call incurs Python interpreter cost.

Comprehensive Comparison

To conduct a thorough empirical evaluation, BFS and DFS were tested across two graph density categories with increasing vertex counts. The comparison focuses on prac-

tical runtime differences between the two traversal strategies under controlled conditions.

- **Sparse graphs** (density $\approx 20\%$): A limited number of edges relative to the maximum possible; represents real-world networks such as road maps or social graphs.
- **Dense graphs** (density $\approx 80\%$): A large number of edges approaching the theoretical maximum; stresses both traversal algorithms and memory access patterns.

BFS vs DFS — Sparse Graphs

The following table presents the measured execution times of BFS and DFS on sparse graphs (density = 0.2) for each tested vertex count:

Density	Vertices	BFS (s)	DFS (s)
0.2	10	0.000011	0.000003
0.2	50	0.000011	0.000011
0.2	100	0.000036	0.000041
0.2	500	0.001155	0.001013
0.2	1000	0.004315	0.005177
0.2	2000	0.052861	0.048143

Figure 7: BFS vs DFS execution times on sparse graphs (density = 0.2)

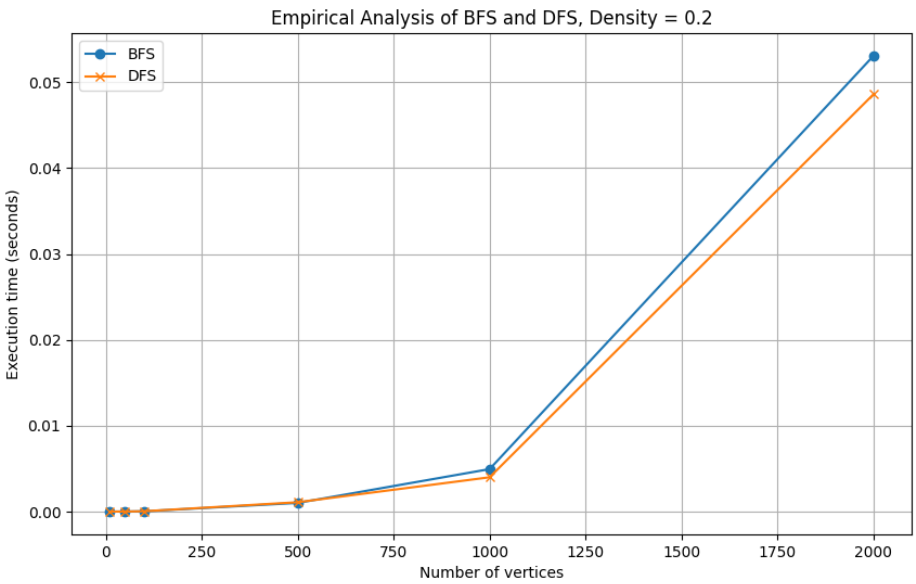

Figure 8: BFS and DFS execution time on sparse graphs (density = 0.2)

On sparse graphs, both algorithms perform comparably. The small number of edges limits the total work per vertex, and any constant-factor differences between queue-based and recursive approaches are minimal at this scale.

BFS vs DFS — Dense Graphs

The following table presents the measured execution times of BFS and DFS on dense graphs (density = 0.8) for each tested vertex count:

Density	Vertices	BFS (s)	DFS (s)
0.8	10	0.000010	0.000004
0.8	50	0.000031	0.000032
0.8	100	0.000094	0.000098
0.8	500	0.004054	0.003992
0.8	1000	0.036482	0.038221
0.8	2000	0.261102	0.235565

Figure 9: BFS vs DFS execution times on dense graphs (density = 0.8)

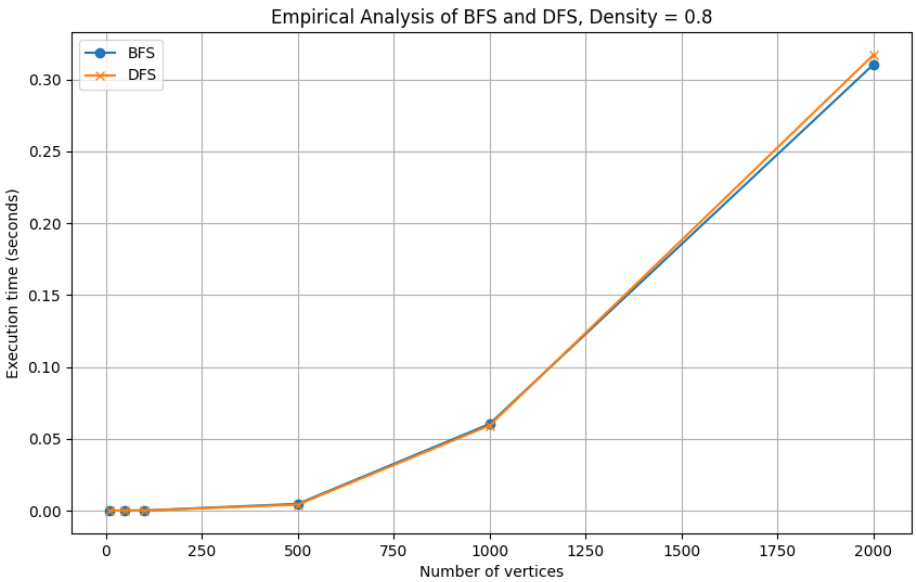

Figure 10: BFS and DFS execution time on dense graphs (density = 0.8)

On dense graphs, the performance gap widens as vertex count increases. DFS tends to incur greater overhead due to deep recursive call stacks, while BFS benefits from iterative queue operations with lower per-step cost in Python.

Key Observations

- **Theoretical equivalence:** Both BFS and DFS share $O(V + E)$ complexity, and this is confirmed empirically — execution time grows proportionally with graph size and density.

- **Practical difference on dense graphs:** DFS is slightly slower on dense graphs due to Python's recursive function call overhead. Each recursive call adds interpreter overhead absent in BFS's iterative loop.

- **Sparse vs. dense behavior:** On sparse graphs, both algorithms are nearly identical in runtime. On dense graphs, differences become noticeable at larger vertex counts.

- **Recursion depth constraint:** DFS required raising Python's default recursion limit (`sys.setrecursionlimit(5000)`) for graphs with 1000+ vertices. An iterative DFS implementation (using an explicit stack) would eliminate this limitation.

- **Disconnected graph handling:** Both implementations correctly handle disconnected graphs by iterating over all vertices and initiating traversal from unvisited ones.

Conclusion

Through empirical analysis, DFS and BFS were evaluated to examine both theoretical time complexity and practical runtime behavior across varying graph sizes and densities.

BFS demonstrates consistent and stable performance due to its iterative, queue-based design. It processes nodes level by level, ensuring that all neighbors at the current depth are explored before advancing. This approach benefits from predictable memory access patterns and avoids the overhead of recursive calls, making it well-suited for large or dense graphs in Python.

DFS achieves equivalent theoretical complexity but shows slightly higher execution times in this recursive Python implementation, particularly on large dense graphs where call stack depth increases significantly. Despite this, DFS remains highly effective for applications such as topological sorting, cycle detection, and generating spanning trees.

Both algorithms correctly handle disconnected graphs by iterating over all vertices. The practical choice between DFS and BFS depends on the use case:

- For **shortest path** problems in unweighted graphs: **BFS** is preferred.

- For **deep exploration**, cycle detection, or topological analysis: **DFS** is preferred.

- For **large dense graphs in Python**: **BFS** avoids recursion depth limitations.
- For **memory-constrained** environments: **DFS** may use less memory on sparse graphs since it does not maintain a frontier queue of wide breadth.

This comparative study demonstrates that while DFS and BFS are theoretically equivalent in time complexity, implementation strategy and graph structure significantly influence practical performance. Empirical analysis complements theoretical reasoning by surfacing these real-world differences.

Git repository: `https://github.com/Adelina1905/Algorithm-Labs`